

NORMALIZATION

523CS0012(Gaurav Singh)

PART A – First Normal Form

The screenshot shows a SQL query editor with three tabs: 523CS0012_LAB02.sql, 523CS0012_LAB03.sql, and 523CS0012_LAB04.sql. The active tab is 523CS0012_LAB02.sql, which contains the following SQL code:

```
--CREATE TABLE ORDERS_RAW(  
--OrderId NUMBER(10),  
--Customer VARCHAR(20),  
--Products VARCHAR(25)  
--);  
--INSERT INTO ORDERS_RAW VALUES(1,'Alice','Pen,Notebook');  
--INSERT INTO ORDERS_RAW VALUES(2,'Bob','Pencil');  
--INSERT INTO ORDERS_RAW VALUES(3,'Alice','Eraser, Pen');  
  
SELECT * FROM ORDERS_RAW;
```

The query result is displayed in a table with the following data:

ORDERID	CUSTOMER	PRODUCTS
1	1 Alice	Pen,Notebook
2	2 Bob	Pencil
3	3 Alice	Eraser, Pen

- The Problem in a Product Column is presence of Multiple values in one row of products column which makes table redundant. Also Its constraint for 1 NF.
- Converted to 1 NF by Creating New table As including Atomic values.

```

--CREATE TABLE ORDERS_RAW(
--OrderId NUMBER(10),
--Customer VARCHAR(20),
--Products VARCHAR(25)
--);
--INSERT INTO ORDERS_RAW VALUES(1,'Alice','Pen,Notebook');
--INSERT INTO ORDERS_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_RAW VALUES(3,'Alice','Eraser, Pen');

--CREATE TABLE ORDERS_N_RAW(
--OrderId NUMBER(10),
--Customer VARCHAR(20),
--Products VARCHAR(25)
--);
--
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Pen');
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Notebook');
--INSERT INTO ORDERS_N_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Eraser');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Pen');

SELECT * FROM ORDERS_N_RAW;

```

Script Output x Query Result x

SQL | All Rows Fetched: 5 in 0.001 seconds

	ORDERID	CUSTOMER	PRODUCTS
1	1	Alice	Pen
2	1	Alice	Notebook
3	2	Bob	Pencil
4	3	Alice	Eraser
5	3	Alice	Pen

PART B – SECOND NORMAL FORM (2 NF)

- The Created Table is as

The screenshot shows a SQL IDE with multiple tabs. The active tab is '523CS0012_LAB05.sql'. The 'Query Builder' tab is selected, showing the following SQL script:

```
--Products VARCHAR(25)
--);
--INSERT INTO ORDERS_RAW VALUES(1,'Alice','Pen,Notebook');
--INSERT INTO ORDERS_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_RAW VALUES(3,'Alice','Eraser, Pen');

--CREATE TABLE ORDERS_N_RAW(
--OrderId NUMBER(10),
--Customer VARCHAR(20),
--Products VARCHAR(25)
--);

--
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Pen');
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Notebook');
--INSERT INTO ORDERS_N_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Eraser');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Pen');
--
--SELECT * FROM ORDERS_N_RAW;

--CREATE TABLE ENROLL(
--StudentId NUMBER(5),
--CourseId VARCHAR(10),
--StudentName VARCHAR(20),
--Marks NUMBER(4)
--);

--INSERT INTO ENROLL VALUES(1,'DBMS','Alice',85);
--INSERT INTO ENROLL VALUES(1,'SQL','Alice',90);
--INSERT INTO ENROLL VALUES(2,'AI','Bob',88);
--INSERT INTO ENROLL VALUES(2,'ML','Bob',92);

SELECT * FROM ENROLL;
```

The 'Script Output' tab is selected, showing the query result:

SQL | All Rows Fetched: 4 in 0.001 seconds

	STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1	DBMS	Alice	85
2	1	SQL	Alice	90
3	2	AI	Bob	88
4	2	ML	Bob	92

- The Primary Key For This table is composite Primary Key which is Primary key(StudentId , CourseId)
- Whenever we are updating name where StudentId = 1 in only row we find that for same studentid we are getting two different students. As here composite key is studentid,courseid.

```

--CREATE TABLE ORDERS_N_RAW(
--OrderId NUMBER(10),
--Customer VARCHAR(20),
--Products VARCHAR(25)
--);
--
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Pen');
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Notebook');
--INSERT INTO ORDERS_N_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Eraser');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Pen');
--
--SELECT * FROM ORDERS_N_RAW;

--CREATE TABLE ENROLL(
--StudentId NUMBER(5),
--CourseId VARCHAR(10),
--StudentName VARCHAR(20),
--Marks NUMBER(4)
--);
--
--INSERT INTO ENROLL VALUES(1,'DBMS','Alice',85);
--INSERT INTO ENROLL VALUES(1,'SQL','Alice',90);
--INSERT INTO ENROLL VALUES(2,'AI','Bob',88);
--INSERT INTO ENROLL VALUES(2,'ML','Bob',92);

--UPDATE ENROLL
--SET StudentName = 'Joy'
--WHERE StudentId = 1;

SELECT * FROM ENROLL;

```

Script Output x Query Result x

SQL | All Rows Fetched: 4 in 0.001 seconds

STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1 DBMS	Joy	85
2	1 SQL	Joy	90
3	2 AI	Bob	88
4	2 ML	Bob	92

- We have created two table to remove dependencies which are required to have table in 2 NF. Table is as Follows

The screenshot shows a SQL IDE with a script editor and a query result window. The script editor contains the following SQL commands:

```
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Pen');
--INSERT INTO ORDERS_N_RAW VALUES(1,'Alice','Notebook');
--INSERT INTO ORDERS_N_RAW VALUES(2,'Bob','Pencil');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Eraser');
--INSERT INTO ORDERS_N_RAW VALUES(3,'Alice','Pen');
--
--SELECT * FROM ORDERS_N_RAW;

--CREATE TABLE ENROLL(
--StudentId NUMBER(5),
--CourseId VARCHAR(10),
--StudentName VARCHAR(20),
--Marks NUMBER(4)
--);

--INSERT INTO ENROLL VALUES(1,'DBMS','Alice',85);
--INSERT INTO ENROLL VALUES(1,'SQL','Alice',90);
--INSERT INTO ENROLL VALUES(2,'AI','Bob',88);
--INSERT INTO ENROLL VALUES(2,'ML','Bob',92);

--UPDATE ENROLL
--SET StudentName = 'Joy'
--WHERE StudentId = 1;

--CREATE TABLE STUDENT_N_ENROLL(
--StudentId INT PRIMARY KEY,
--StudentName VARCHAR(10)
--);
--INSERT INTO STUDENT_N_ENROLL VALUES(1,'Alice');
--INSERT INTO STUDENT_N_ENROLL VALUES(2,'Bob');

SELECT * FROM STUDENT_N_ENROLL;
```

The query result window shows the output of the last query, which is a table with two columns: STUDENTID and STUDENTNAME. The table contains two rows of data:

STUDENTID	STUDENTNAME
1	Alice
2	Bob

...sql 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x

Worksheet Query Builder

```
--Marks NUMBER(4)
--);

--INSERT INTO ENROLL VALUES(1,'DBMS','Alice',85);
--INSERT INTO ENROLL VALUES(1,'SQL','Alice',90);
--INSERT INTO ENROLL VALUES(2,'AI','Bob',88);
--INSERT INTO ENROLL VALUES(2,'ML','Bob',92);

--UPDATE ENROLL
--SET StudentName = 'Joy'
--WHERE StudentId = 1;

--CREATE TABLE STUDENT_N_ENROLL(
--StudentId INT PRIMARY KEY,
--StudentName VARCHAR(10)
--);
--INSERT INTO STUDENT_N_ENROLL VALUES(1,'Alice');
--INSERT INTO STUDENT_N_ENROLL VALUES(2,'Bob');

--CREATE TABLE ENROLL_N(
--StudentId INT,
--CourseId VARCHAR(10),
--Marks NUMBER(4),
--PRIMARY KEY(StudentId,CourseId),
--FOREIGN KEY (StudentId) REFERENCES STUDENT_N_ENROLL(StudentId)
--);

--INSERT INTO ENROLL_N VALUES(1,'DBMS',85);
--INSERT INTO ENROLL_N VALUES(1,'SQL',90);
--INSERT INTO ENROLL_N VALUES(2,'AI',88);
--INSERT INTO ENROLL_N VALUES(2,'ML',92);

SELECT * FROM ENROLL_N;
```

Script Output x Query Result x

SQL | All Rows Fetched: 4 in 0.001 seconds

	STUDENTID	COURSEID	MARKS
1	1	DBMS	85
2	1	SQL	90
3	2	AI	88
4	2	ML	92

PART C – Third Normal Form(3 - NF)

- Initial Table is as

```
--CREATE TABLE STUDENT_N_ENROLL(  
--StudentId INT PRIMARY KEY,  
--StudentName VARCHAR(10)  
--);  
--INSERT INTO STUDENT_N_ENROLL VALUES(1,'Alice');  
--INSERT INTO STUDENT_N_ENROLL VALUES(2,'Bob');  
  
--CREATE TABLE ENROLL_N(  
--StudentId INT,  
--CourseId VARCHAR(10),  
--Marks NUMBER(4),  
--PRIMARY KEY(StudentId,CourseId),  
--FOREIGN KEY (StudentId) REFERENCES STUDENT_N_ENROLL(StudentId)  
--);  
  
--INSERT INTO ENROLL_N VALUES(1,'DBMS',85);  
--INSERT INTO ENROLL_N VALUES(1,'SQL',90);  
--INSERT INTO ENROLL_N VALUES(2,'AI',88);  
--INSERT INTO ENROLL_N VALUES(2,'ML',92);  
  
--  
--SELECT * FROM ENROLL_N;  
  
--CREATE TABLE EMPLOYEE(  
--EmpId NUMBER(5),  
--EmpName VARCHAR(20),  
--DeptId VARCHAR(5),  
--DeptName VARCHAR(20)  
--);  
  
--  
--INSERT INTO EMPLOYEE VALUES(101,'Anita','D1','HR');  
--INSERT INTO EMPLOYEE VALUES(102,'Meena','D2','IT');  
--INSERT INTO EMPLOYEE VALUES(103,'Rakesh','D1','HR');  
--  
SELECT * FROM EMPLOYEE;
```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.001 seconds

	EMPID	EMPNAME	DEPTID	DEPTNAME
1	101	Anita	D1	HR
2	102	Meena	D2	IT
3	103	Rakesh	D1	HR

- Updating DeptName where deptid = D1

The screenshot shows a SQL IDE with a script editor and a results panel. The script contains SQL commands to create two tables, insert data, and update a record. The results panel shows the output of a SELECT query on the EMPLOYEE table.

```
--CREATE TABLE ENROLL_N(
--StudentId INT,
--CourseId VARCHAR(10),
--Marks NUMBER(4),
--PRIMARY KEY(StudentId,CourseId),
--FOREIGN KEY (StudentId) REFERENCES STUDENT_N_ENROLL(StudentId)
--);

--INSERT INTO ENROLL_N VALUES(1,'DBMS',85);
--INSERT INTO ENROLL_N VALUES(1,'SQL',90);
--INSERT INTO ENROLL_N VALUES(2,'AI',88);
--INSERT INTO ENROLL_N VALUES(2,'ML',92);

--
--SELECT * FROM ENROLL_N;

--CREATE TABLE EMPLOYEE(
--EmpId NUMBER(5),
--EmpName VARCHAR(20),
--DeptId VARCHAR(5),
--DeptName VARCHAR(20)
--);

--
--INSERT INTO EMPLOYEE VALUES(101,'Anita','D1','HR');
--INSERT INTO EMPLOYEE VALUES(102,'Meena','D2','IT');
--INSERT INTO EMPLOYEE VALUES(103,'Rakesh','D1','HR');
--
--UPDATE EMPLOYEE
--SET DeptName = 'CS'
--WHERE DeptId = 'D1' AND EmpId = 101;

--SELECT * FROM EMPLOYEE;
```

The results panel shows the following data:

EMPID	EMPNAME	DEPTID	DEPTNAME
1	101 Anita	D1	CS
2	102 Meena	D2	IT
3	103 Rakesh	D1	HR

- We are getting Transitive Dependency as Non Prime attribute is determining Non Prime attribute which is deptid -> deptname
- Table After converting to 3 NF----

The screenshot shows a SQL IDE window with multiple tabs. The active tab is '523CS0012_LAB02.sql'. The script contains the following SQL statements:

```
--INSERT INTO ENROLL_N VALUES(2, 'AI', 88);
--INSERT INTO ENROLL_N VALUES(2, 'ML', 92);

--
--SELECT * FROM ENROLL_N;

--CREATE TABLE EMPLOYEE(
--EmpId NUMBER(5),
--EmpName VARCHAR(20),
--DeptId VARCHAR(5),
--DeptName VARCHAR(20)
--);

--
--INSERT INTO EMPLOYEE VALUES(101, 'Anita', 'D1', 'HR');
--INSERT INTO EMPLOYEE VALUES(102, 'Meena', 'D2', 'IT');
--INSERT INTO EMPLOYEE VALUES(103, 'Rakesh', 'D1', 'HR');
--
--UPDATE EMPLOYEE
--SET DeptName = 'CS'
--WHERE DeptId = 'D1' AND EmpId = 101;

--CREATE TABLE EMPLOYEE_R(
--DeptId VARCHAR(10) PRIMARY KEY,
--DeptName VARCHAR(15)
--);

--INSERT INTO EMPLOYEE_R VALUES('D1', 'HR');
--INSERT INTO EMPLOYEE_R VALUES('D2', 'IT');
--
--CREATE TABLE EMPLOYEE_R(
--DeptId VARCHAR(10) PRIMARY KEY,
--DeptName VARCHAR(15)
--);

SELECT * FROM EMPLOYEE_R;
```

The 'Query Result' tab shows the following data:

	DEPTID	DEPTNAME
1	D1	HR
2	D2	IT

...sql 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x

Worksheet Query Builder

```
--EmpId NUMBER(5),
--EmpName VARCHAR(20),
--DeptId VARCHAR(5),
--DeptName VARCHAR(20)
--);

--
--INSERT INTO EMPLOYEE VALUES(101,'Anita','D1','HR');
--INSERT INTO EMPLOYEE VALUES(102,'Meena','D2','IT');
--INSERT INTO EMPLOYEE VALUES(103,'Rakesh','D1','HR');
--
--UPDATE EMPLOYEE
--SET DeptName = 'CS'
--WHERE DeptId = 'D1' AND EmpId = 101;

--CREATE TABLE EMPLOYEE_R(
--DeptId VARCHAR(10) PRIMARY KEY,
--DeptName VARCHAR(15)
--);

--INSERT INTO EMPLOYEE_R VALUES('D1','HR');
--INSERT INTO EMPLOYEE_R VALUES('D2','IT');
--
--CREATE TABLE EMPLOYEE_2R(
--EmpId INT PRIMARY KEY,
--EmpName VARCHAR(15),
--DeptId VARCHAR(10),
--FOREIGN KEY (DeptId) REFERENCES EMPLOYEE_R(DeptId)
--);

--INSERT INTO EMPLOYEE_2R VALUES(101,'Anita','D1');
--INSERT INTO EMPLOYEE_2R VALUES(102,'Meena','D2');
--INSERT INTO EMPLOYEE_2R VALUES(103,'Rakesh','D1');

SELECT * FROM EMPLOYEE_2R;
```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.001 seconds

	EMPID	EMPNAME	DEPTID
1	101	Anita	D1
2	102	Meena	D2
3	103	Rakesh	D1

PART D – Boyce – Codd Normal Form (BCNF)

- Initial Table

```
--SET DeptName = 'CS'
--WHERE DeptId = 'D1' AND EmpId = 101;

--CREATE TABLE EMPLOYEE_R(
--DeptId VARCHAR(10) PRIMARY KEY,
--DeptName VARCHAR(15)
--);

--INSERT INTO EMPLOYEE_R VALUES('D1','HR');
--INSERT INTO EMPLOYEE_R VALUES('D2','IT');
--
--CREATE TABLE EMPLOYEE_2R(
--EmpId INT PRIMARY KEY,
--EmpName VARCHAR(15),
--DeptId VARCHAR(10),
--FOREIGN KEY (DeptId) REFERENCES EMPLOYEE_R(DeptId)
--);

--INSERT INTO EMPLOYEE_2R VALUES(101,'Anita','D1');
--INSERT INTO EMPLOYEE_2R VALUES(102,'Meena','D2');
--INSERT INTO EMPLOYEE_2R VALUES(103,'Rakesh','D1');

--SELECT * FROM EMPLOYEE_2R;

--CREATE TABLE TEACHING(
--Professor VARCHAR(20),
--Subject VARCHAR(10),
--Room VARCHAR(10)
--);

--INSERT INTO TEACHING VALUES('Dr Rao','DBMS','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Rao','SQL','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Meena','AI','ROOM202');
--INSERT INTO TEACHING VALUES('Dr Meena','ML','ROOM202');

SELECT * FROM TEACHING;
```

Script Output x Query Result x

SQL | All Rows Fetched: 4 in 0.001 seconds

	PROFESSOR	SUBJECT	ROOM
1	Dr Rao	DBMS	ROOM101
2	Dr Rao	SQL	ROOM101
3	Dr Meena	AI	ROOM202
4	Dr Meena	ML	ROOM202

- Updated Room of one professor

The screenshot shows a SQL IDE with a script editor and a query result window. The script editor contains the following SQL code:

```
--CREATE TABLE TEACHING(
--Professor VARCHAR(20),
--Subject VARCHAR(10),
--Room VARCHAR(10)
--);

--INSERT INTO TEACHING VALUES('Dr Rao','DBMS','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Rao','SQL','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Meena','AI','ROOM202');
--INSERT INTO TEACHING VALUES('Dr Meena','ML','ROOM202');

--UPDATE TEACHING
--SET Room = 'Room501'
--WHERE Subject = 'DBMS';

--CREATE TABLE TEACHING_R(
--Professor VARCHAR(20) PRIMARY KEY,
--Room VARCHAR(10)
--);

--INSERT INTO TEACHING_R VALUES('Dr Rao','ROOM101');
--INSERT INTO TEACHING_R VALUES('Dr Meena','ROOM202');

--CREATE TABLE TEACHING_2R(
--Professor VARCHAR(20),
--Subject VARCHAR(10),
--PRIMARY KEY(Professor,Subject),
--FOREIGN KEY (Professor) REFERENCES TEACHING_R(Professor)
--);

--INSERT INTO TEACHING_2R VALUES('Dr Rao','DBMS');
--INSERT INTO TEACHING_2R VALUES('Dr Rao','SQL');
--INSERT INTO TEACHING_2R VALUES('Dr Meena','AI');
--INSERT INTO TEACHING_2R VALUES('Dr Meena','ML');

SELECT * FROM TEACHING;
```

The query result window shows the following data:

	PROFESSOR	SUBJECT	ROOM
1	Dr Rao	DBMS	Room501
2	Dr Rao	SQL	ROOM101
3	Dr Meena	AI	ROOM202
4	Dr Meena	ML	ROOM202

- We noticed that for same professor we have two rooms.

- Professor -> room violates BCNF

...sql 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x

Worksheet Query Builder

```
--EmpId INT PRIMARY KEY,
--EmpName VARCHAR(15),
--DeptId VARCHAR(10),
--FOREIGN KEY (DeptId) REFERENCES EMPLOYEE_R(DeptId)
--);

--INSERT INTO EMPLOYEE_2R VALUES(101,'Anita','D1');
--INSERT INTO EMPLOYEE_2R VALUES(102,'Meena','D2');
--INSERT INTO EMPLOYEE_2R VALUES(103,'Rakesh','D1');

--SELECT * FROM EMPLOYEE_2R;

--CREATE TABLE TEACHING(
--Professor VARCHAR(20),
--Subject VARCHAR(10),
--Room VARCHAR(10)
--);

--INSERT INTO TEACHING VALUES('Dr Rao','DBMS','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Rao','SQL','ROOM101');
--INSERT INTO TEACHING VALUES('Dr Meena','AI','ROOM202');
--INSERT INTO TEACHING VALUES('Dr Meena','ML','ROOM202');

--UPDATE TEACHING
--SET Room = 'Room501'
--WHERE Subject = 'DBMS';

--CREATE TABLE TEACHING_R(
--Professor VARCHAR(20) PRIMARY KEY,
--Room VARCHAR(10)
--);

--INSERT INTO TEACHING_R VALUES('Dr Rao','ROOM101');
--INSERT INTO TEACHING_R VALUES('Dr Meena','ROOM202');

SELECT * FROM TEACHING_R;
```

Script Output x Query Result x

SQL | All Rows Fetched: 2 in 0.001 seconds

	PROFESSOR	ROOM
1	Dr Meena	ROOM202
2	Dr Rao	ROOM101

...sql 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x

Worksheet Query Builder

```
--CREATE TABLE TEACHING(  
--Professor VARCHAR(20),  
--Subject VARCHAR(10),  
--Room VARCHAR(10)  
--);  
  
--INSERT INTO TEACHING VALUES('Dr Rao','DBMS','ROOM101');  
--INSERT INTO TEACHING VALUES('Dr Rao','SQL','ROOM101');  
--INSERT INTO TEACHING VALUES('Dr Meena','AI','ROOM202');  
--INSERT INTO TEACHING VALUES('Dr Meena','ML','ROOM202');  
  
--UPDATE TEACHING  
--SET Room = 'Room501'  
--WHERE Subject = 'DBMS';  
  
--CREATE TABLE TEACHING_R(  
--Professor VARCHAR(20) PRIMARY KEY,  
--Room VARCHAR(10)  
--);  
  
--INSERT INTO TEACHING_R VALUES('Dr Rao','ROOM101');  
--INSERT INTO TEACHING_R VALUES('Dr Meena','ROOM202');  
  
--CREATE TABLE TEACHING_2R(  
--Professor VARCHAR(20),  
--Subject VARCHAR(10),  
--PRIMARY KEY(Professor,Subject),  
--FOREIGN KEY (Professor) REFERENCES TEACHING_R(Professor)  
--);  
--INSERT INTO TEACHING_2R VALUES('Dr Rao','DBMS');  
--INSERT INTO TEACHING_2R VALUES('Dr Rao','SQL');  
--INSERT INTO TEACHING_2R VALUES('Dr Meena','AI');  
--INSERT INTO TEACHING_2R VALUES('Dr Meena','ML');  
  
SELECT * FROM TEACHING_2R;
```

Script Output x Query Result x

SQL | All Rows Fetched: 4 in 0.001 seconds

	PROFESSOR	SUBJECT
1	Dr Rao	DBMS
2	Dr Rao	SQL
3	Dr Meena	AI
4	Dr Meena	ML

- After Removing dependency professor $\rightarrow$ room.
- This is removed by creating two relations one itself has primary key professor and other is having composite primary key. This Makes LHS of every functional dependency is a super key.